

Circuit Breaker + Retry Setup Guide - Spring Boot Microservice

Circuit Breaker + Retry Setup Guide — Java Spring Boot Microservice (Resilience4j)

Complete, working, step-by-step setup — from a blank project to a tuned, production-ready resilience configuration with correct Circuit Breaker + Retry + Bulkhead ordering.

Step 1 — Add Dependencies

Maven (pom.xml)

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Resilience4j Spring Boot integration -->
  <dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-boot3</artifactId>
    <version>2.2.0</version>
  </dependency>

  <!-- Required for @CircuitBreaker/@Retry AOP proxying -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
  </dependency>

  <!-- Actuator – exposes circuit breaker health/metrics endpoints -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>

  <!-- Optional but recommended – feeds Resilience4j metrics into
  Actuator/Prometheus -->
  <dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
  </dependency>
</dependencies>
```

Gradle (build.gradle)

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-
web'
    implementation 'io.github.resilience4j:resilience4j-spring-
boot3:2.2.0'
    implementation 'org.springframework.boot:spring-boot-starter-
```

```

aop'
    implementation 'org.springframework.boot:spring-boot-starter-
actuator'
    implementation 'io.micrometer:micrometer-registry-prometheus'
}

```

Use resilience4j-spring-boot3 for Spring Boot 3.x (Spring Framework 6, Jakarta namespace). If you're on Spring Boot 2.x, use resilience4j-spring-boot2 instead.

Step 2 — Configure Resilience4j in application.yml

This sets up Circuit Breaker, Retry, and Bulkhead together, with settings tuned to avoid the retry-storm problem discussed earlier (jitter, capped half-open traffic).

```

resilience4j:
  circuitbreaker:
    instances:
      inventoryService:
        register-health-indicator: true
        sliding-window-type: COUNT_BASED      # or TIME_BASED for
time-windowed evaluation
        sliding-window-size: 20                # evaluate the last
20 calls
        minimum-number-of-calls: 10           # don't trip until at
least 10 calls have happened
        failure-rate-threshold: 50            # open if >= 50% of
those calls failed
        wait-duration-in-open-state: 10s      # stay OPEN for 10s
before testing recovery
        permitted-number-of-calls-in-half-open-state: 3  # cap test
traffic in HALF_OPEN
        automatic-transition-from-open-to-half-open-enabled: true
        slow-call-duration-threshold: 2s      # calls slower than
this count as "slow"
        slow-call-rate-threshold: 50          # too many slow
calls also trips the breaker

  retry:
    instances:
      inventoryService:
        max-attempts: 3
        wait-duration: 500ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        retry-exceptions:
          - java.io.IOException
          - org.springframework.web.client.ResourceAccessException
        ignore-exceptions:
          -
com.example.orderservice.exception.BusinessValidationException #
don't retry non-transient errors

  bulkhead:
    instances:
      inventoryService:
        max-concurrent-calls: 10
        max-wait-duration: 0

  thread-pool-bulkhead:
    instances:
      inventoryService:
        max-thread-pool-size: 10
        core-thread-pool-size: 5

```

```

        queue-capacity: 20

management:
  endpoints:
    web:
      exposure:
        include: health, circuitbreakers, retries, metrics,
prometheus
  endpoint:
    health:
      show-details: always
  health:
    circuitbreakers:
      enabled: true

```

Jitter isn't a plain YAML property — configure it in code (Step 5) using `IntervalFunction.ofExponentialRandomBackoff(...)`, since the declarative YAML config only supports fixed exponential backoff, not randomized jitter.

Step 3 — Define the Downstream Client Call

```

@Component
public class InventoryClient {

    private final RestClient restClient;

    public InventoryClient(RestClient.Builder builder) {
        this.restClient = builder.baseUrl("http://inventory-
service").build();
    }

    public Inventory getStock(String skuId) {
        return restClient.get()
            .uri("/stock/{sku}", skuId)
            .retrieve()
            .body(Inventory.class);
    }
}

```

Step 4 — Apply Circuit Breaker + Retry + Bulkhead With Correct Ordering

Critical: annotation order determines decorator nesting. Circuit Breaker must be the outermost annotation so it evaluates the *net* outcome of all retries as a single call — not each individual attempt.

```

@Service
public class InventoryClientService {

    private final InventoryClient inventoryClient;

    public InventoryClientService(InventoryClient inventoryClient) {
        this.inventoryClient = inventoryClient;
    }

    @CircuitBreaker(name = "inventoryService", fallbackMethod =
"fallback")    // OUTERMOST
    @Bulkhead(name = "inventoryService")
    // MIDDLE
    @Retry(name = "inventoryService")
    // INNERMOST – closest to the real call

```

```

        public Inventory checkStock(String skuId) {
            return inventoryClient.getStock(skuId);
        }

        // Fallback signature must match the original method + a
        // Throwable parameter
        public Inventory fallback(String skuId, Throwable t) {
            return Inventory.unavailable(skuId); // graceful degradation
        }
    }
}

```

Why this order: Retry attempts the call up to 3 times *before* Circuit Breaker ever sees a result. Circuit Breaker then counts that as **one** success/failure for its failure-rate calculation — not three — so retries don't artificially inflate the breaker's failure count and trip it prematurely. Bulkhead sits in between, capping concurrent in-flight calls (including retries) to this dependency specifically.

Step 5 — Add Jitter to Retry Backoff (Code-Based Config)

```

@Configuration
public class ResilienceConfig {

    @Bean
    public RetryConfigCustomizer inventoryServiceRetryCustomizer() {
        return RetryConfigCustomizer.of("inventoryService", builder
->
            builder.intervalFunction(
                IntervalFunction.ofExponentialRandomBackoff(
                    Duration.ofMillis(500), // initial wait
                    2.0, // multiplier
                    0.5 // randomization factor
                )
            )
        );
    }
}

```

(jitter) – spreads retries out

This prevents synchronized retry storms — instead of every failing client retrying at exactly 500ms/1000ms/2000ms in lockstep, the jitter factor spreads retries across a randomized window, reducing the chance that a recovering dependency gets hit by a coordinated wave of retries right as it comes back up.

Step 6 — Register a State-Change Listener (Observability)

```

@Configuration
public class CircuitBreakerEventConfig {

    private static final Logger log =
    LoggerFactory.getLogger(CircuitBreakerEventConfig.class);

    @Bean
    public RegistryEventConsumer<CircuitBreaker>
    circuitBreakerEventConsumer() {
        return new RegistryEventConsumer<>() {
            @Override
            public void
            onEntryAddedEvent(EntryAddedEvent<CircuitBreaker> event) {
                event.getAddedEntry().getEventPublisher()

```

```

        .onStateTransition(e ->
            log.warn("Circuit breaker '{}' transitioned
from {} to {}",
                    e.getCircuitBreakerName(),
                    e.getStateTransition().getFromState(),
                    e.getStateTransition().getToState()));
    }
    @Override public void
onEntryRemovedEvent(EntryRemovedEvent<CircuitBreaker> event) { }
    @Override public void
onEntryReplacedEvent(EntryReplacedEvent<CircuitBreaker> event) { }
    };
    }
}

```

This logs every CLOSED → OPEN, OPEN → HALF_OPEN, and HALF_OPEN → CLOSED/OPEN transition — wire this into your alerting (e.g., forward to Slack/PagerDuty) so the team knows immediately when a dependency starts failing at scale, rather than discovering it from a spike in fallback responses.

Step 7 — Verify via Actuator Endpoints

Once the app is running:

```

curl http://localhost:8080/actuator/health
curl http://localhost:8080/actuator/circuitbreakers
curl http://localhost:8080/actuator/retries
curl
http://localhost:8080/actuator/metrics/resilience4j.circuitbreaker.calls

/actuator/health shows the circuit breaker's current state
(UP/DOWN/CIRCUIT_OPEN depending on show-details), and
/actuator/circuitbreakers returns full configuration + current metrics
(failure rate, number of calls, state) for every registered breaker
instance.

```

Step 8 — Write a Test That Forces the Breaker Open

```

@SpringBootTest
class InventoryClientServiceTest {

    @Autowired
    private InventoryClientService inventoryClientService;

    @Autowired
    private CircuitBreakerRegistry circuitBreakerRegistry;

    @MockBean
    private InventoryClient inventoryClient;

    @Test
    void circuitOpensAfterRepeatedFailures() {
        when(inventoryClient.getStock(anyString()))
            .thenReturn(new ResourceAccessException("Connection
refused"));

        // minimum-number-of-calls is 10, so trigger at least that
many failures
        for (int i = 0; i < 12; i++) {
            inventoryClientService.checkStock("SKU-1"); // each call
returns the fallback, doesn't throw
        }
    }
}

```

```

        CircuitBreaker cb =
circuitBreakerRegistry.circuitBreaker("inventoryService");

assertThat(cb.getState()).isEqualTo(CircuitBreaker.State.OPEN);
    }
}

```

Step 9 — Tune for Real Production Traffic (Senior-Level Checklist)

Setting	What to consider
sliding-window-type	COUNT_BASED for steady traffic; TIME_BASED for bursty/low-traffic services where a fixed call count takes too long to fill
minimum-number-of-calls	Too low → trips on noise from a handful of unlucky calls; too high → slow to react to a real outage
failure-rate-threshold	Balance false positives (too sensitive) vs slow detection (too lenient) — start around 50%, adjust from real incident data
wait-duration-in-open-state	Should roughly match how long the dependency typically takes to recover from a blip
retry-exceptions / ignore-exceptions	Only retry genuinely transient failures (timeouts, connection resets) — never retry validation errors, 4xx client errors, or non-idempotent business failures
Retry + idempotency	Confirm the wrapped operation is safe to call multiple times before enabling Retry at all (see Idempotent Consumer pattern for write operations)
Bulkhead size	Set based on the dependency's known safe concurrency limit, not a guess — undersized bulkheads cause unnecessary rejections, oversized ones don't protect anything

Full Working Flow Summary

```

Client call
↓
@CircuitBreaker → checks state; if OPEN, fails fast to fallback() immediately
    ↓ (if CLOSED or HALF_OPEN, permitted)
@Bulkhead → checks concurrent call limit; rejects if saturated
    ↓ (if capacity available)
@Retry → attempts the real call up to max-attempts times, with jittered backoff
    ↓
InventoryClient.getStock() → actual HTTP call to the downstream service
    ↓
Result bubbles back up: Retry returns net outcome → Bulkhead releases the slot →

```

Circuit Breaker records ONE success/failure → state updated if threshold crossed

This is the complete, correctly-ordered, production-tuned setup: dependencies in place, config tuned against real failure patterns, jitter preventing retry storms, observability wired in, and a test proving the breaker actually trips as designed.